Database System Engineering and Distributed Backend Development

Course Code: 25CS1302E

2520030078

M.Apoorva

Section – 3

Triggers And Stored Procedures

Create database:

```
CREATE DATABASE IF NOT EXISTS bookflow8_db;
```

Output:

```
1  16:48:15  CREATE DATABASE IF NOT EXISTS bookflow8_db
```

Use database:

```
USE bookflow8_db;
```

Output:

```
2  16:49:23  USE bookflow8_db
```

Create Customer Table:

```
CREATE TABLE Customer (
    Customer_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(15),
    Email VARCHAR(100),
    City VARCHAR(50)
);
```

Output:

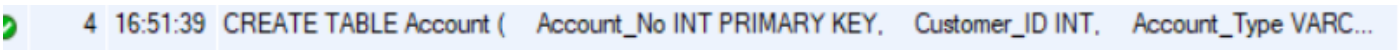
```
3  16:50:34  CREATE TABLE Customer (    Customer_ID INT PRIMARY KEY,    Customer_Name VARCHAR(100) NOT NU.
```

Create Account Table:

```sql
CREATE TABLE Account (
    Account_No INT PRIMARY KEY,
    Customer_ID INT,
    Account_Type VARCHAR(20),
    Balance DECIMAL(12,2) DEFAULT 0,
    Branch VARCHAR(50),

    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID)
);
```

Output:



Create Transaction Table:

```sql
CREATE TABLE Bank_Transaction (
    Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Transaction_Date DATETIME DEFAULT CURRENT_TIMESTAMP,

    FOREIGN KEY (Account_No)
    REFERENCES Account(Account_No)
);
```

Output:

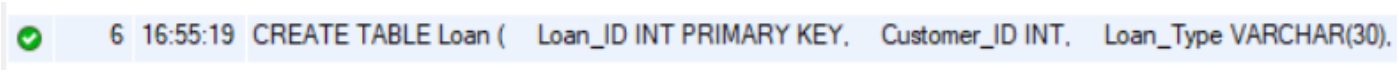


Create loan table:

```sql
CREATE TABLE Loan (
    Loan_ID INT PRIMARY KEY,
    Customer_ID INT,
    Loan_Type VARCHAR(30),
    Loan_Amount DECIMAL(12,2),
    Interest_Rate DECIMAL(5,2),

    FOREIGN KEY (Customer_ID)
    REFERENCES Customer(Customer_ID)
);
```

Output:



Insert Customer Data

```sql
INSERT INTO Customer
(Customer_ID, Customer_Name, Phone, Email, City)
VALUES
(101, 'Sushma', '9876543210', 'ravi@gmail.com', 'Hyderabad'),
(102, 'Sowmya', '9876543211', 'priya@gmail.com', 'Vijayawada'),
(103, 'Mokshagna', '9876543212', 'arjun@gmail.com', 'Bangalore'),
(104, 'Krishna', '9876543213', 'sneha@gmail.com', 'Chennai'),
(105, 'Govinda', '9876543214', 'kiran@gmail.com', 'Hyderabad');
SELECT * FROM Customer;
```

Output:

Customer_ID	Customer_Name	Phone	Email	City
101	Sushma	9876543210	ravi@gmail.com	Hyderabad
102	Sowmya	9876543211	priya@gmail.com	Vijayawada
103	Mokshagna	9876543212	arjun@gmail.com	Bangalore
104	Krishna	9876543213	sneha@gmail.com	Chennai
105	Govinda	9876543214	kiran@gmail.com	Hyderabad
NULL	NULL	NULL	NULL	NULL

Insert Account Data:

```
INSERT INTO Account
(Account_No, Customer_ID, Account_Type, Balance, Branch)
VALUES
(10001, 101, 'Savings', 50000, 'Hyderabad'),
(10002, 102, 'Savings', 75000, 'Vijayawada'),
(10003, 103, 'Current', 120000, 'Bangalore'),
(10004, 104, 'Savings', 45000, 'Chennai'),
(10005, 105, 'Current', 90000, 'Hyderabad');
SELECT * FROM Account;
```

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad
10002	102	Savings	75000.00	Vijayawada
10003	103	Current	120000.00	Bangalore
10004	104	Savings	45000.00	Chennai
10005	105	Current	90000.00	Hyderabad
NULL	NULL	NULL	NULL	NULL

Insert Transaction Data

```
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 10000),
(10002, 'DEPOSIT', 15000),
(10003, 'WITHDRAW', 20000),
(10004, 'DEPOSIT', 5000),
(10005, 'WITHDRAW', 10000);
SELECT * FROM Bank_Transaction;
```

Output:

Transaction_ID	Account_No	Transaction_Type	Amount	Transaction_Date
1	10001	DEPOSIT	10000.00	2026-08-14 17:00:37
2	10002	DEPOSIT	15000.00	2026-08-14 17:00:37
3	10003	WITHDRAW	20000.00	2026-08-14 17:00:37
4	10004	DEPOSIT	5000.00	2026-08-14 17:00:37
5	10005	WITHDRAW	10000.00	2026-08-14 17:00:37
NULL	NULL	NULL	NULL	NULL

Insert Loan Data:

```
INSERT INTO Loan
(Loan_ID, Customer_ID, Loan_Type, Loan_Amount, Interest_Rate)
VALUES
(501, 101, 'Home Loan', 5000000, 7.5),
(502, 102, 'Education Loan', 1000000, 6.5),
(503, 103, 'Car Loan', 800000, 8.2),
(504, 104, 'Personal Loan', 500000, 10.5);
SELECT * FROM Loan;
```

Output:

Loan_ID	Customer_ID	Loan_Type	Loan_Amount	Interest_Rate
501	101	Home Loan	5000000.00	7.50
502	102	Education Loan	1000000.00	6.50
503	103	Car Loan	800000.00	8.20
504	104	Personal Loan	500000.00	10.50
NULL	NULL	NULL	NULL	NULL

Procedure To Display All Customers:

```
DELIMITER //
CREATE PROCEDURE GetAllCustomers()
BEGIN
    SELECT * FROM Customer;
END //
DELIMITER ;
CALL GetAllCustomers();
```

Output:

Customer_ID	Customer_Name	Phone	Email	City
101	Sushma	9876543210	ravi@gmail.com	Hyderabad
102	Sowmya	9876543211	priya@gmail.com	Vijayawada
103	Mokshagna	9876543212	arjun@gmail.com	Bangalore
104	Krishna	9876543213	sneha@gmail.com	Chennai
105	Govinda	9876543214	kiran@gmail.com	Hyderabad

Procedure to display account details

for a given account number:

```
DELIMITER //
CREATE PROCEDURE GetAccountDetails(
    IN p_Account_No INT
)
BEGIN
    SELECT *
    FROM Account
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;
CALL GetAccountDetails(10001);
```

Output:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	50000.00	Hyderabad

Procedure To Find Customer Accounts:

```
DELIMITER //
CREATE PROCEDURE GetCustomerAccounts(
    IN p_Customer_ID INT
)
BEGIN
    SELECT
        C.Customer_ID,
        C.Customer_Name,
        A.Account_No,
        A.Account_Type,
        A.Balance,
        A.Branch
    FROM Customer C
    JOIN Account A
    ON C.Customer_ID = A.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
END //
DELIMITER ;
CALL GetCustomerAccounts(101);
```

Output:

	Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
▶	101	Sushma	10001	Savings	50000.00	Hyderabad

Procedure To Deposit Money

```
DELIMITER //
CREATE PROCEDURE DepositMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance + p_Amount
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;
CALL DepositMoney(10001, 5000);
```

Output:

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Sushma	10001	Savings	50000.00	Hyderabad

Procedure To Withdraw Money

```
DELIMITER //
CREATE PROCEDURE WithdrawMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance - p_Amount
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;
CALL WithdrawMoney(10001, 3000);
```

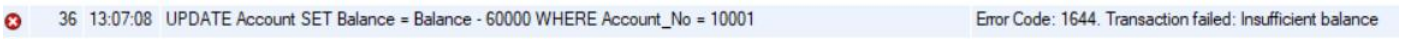
Output:

28	13:00:07	CREATE PROCEDURE WithdrawMoney(	IN p_Account_No INT,	IN p_Amount DECIMAL(12,2)) BEGIN ...	0
29	13:00:10	CALL WithdrawMoney(10001, 3000)			1

Trigger – Prevent Negative Balance:

```
DELIMITER //
CREATE TRIGGER CheckBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction failed: Insufficient balance';
    END IF;
END //
DELIMITER ;
UPDATE Account
SET Balance = Balance - 60000
WHERE Account_No = 10001;
```

Output:



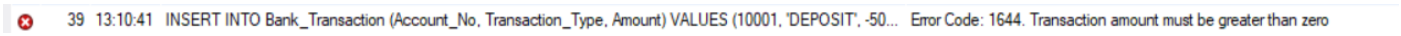
Trigger – Prevent Negative Deposit

```
DELIMITER //
CREATE TRIGGER CheckTransactionAmount
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction amount must be greater than zero';
    END IF;
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', -5000);
```

Output:


```
DELIMITER //
CREATE TRIGGER CheckTransactionAmount
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Amount <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction amount must be greater than zero';
    END IF;
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', -5000);
```

Output:



Create Transaction Audit Table

```
CREATE TABLE Transaction_Audit (
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Transaction_ID INT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Output:



Trigger – Transaction Audit:

```
DELIMITER //
CREATE TRIGGER TransactionAudit
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    INSERT INTO Transaction_Audit
    (
        Transaction_ID,
        Account_No,
        Transaction_Type,
        Amount
    )
    VALUES
    (
        NEW.Transaction_ID,
        NEW.Account_No,
        NEW.Transaction_Type,
        NEW.Amount
    );
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 2500);
SELECT * FROM Transaction_Audit;
```

Output:

	Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
*	NULL	NULL	NULL	NULL	NULL	NULL

Trigger – Automatically Update Account Balance

```
DELIMITER //
CREATE TRIGGER UpdateBalanceAfterTransaction
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Transaction_Type = 'DEPOSIT' THEN
        UPDATE Account
        SET Balance = Balance + NEW.Amount
        WHERE Account_No = NEW.Account_No;
    ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN
        UPDATE Account
        SET Balance = Balance - NEW.Amount
        WHERE Account_No = NEW.Account_No;
    END IF;
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 5000);
SELECT *
FROM Account
WHERE Account_No = 10001;
```

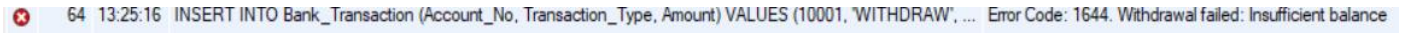
Output:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	43000.00	Hyderabad
*	NULL	NULL	NULL	NULL	NULL

Trigger – Prevent Withdrawal Above Balance

```
DELIMITER //
CREATE TRIGGER PreventInsufficientWithdrawal
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    DECLARE CurrentBalance DECIMAL(12,2);
    SELECT Balance
    INTO CurrentBalance
    FROM Account
    WHERE Account_No = NEW.Account_No;
    IF NEW.Transaction_Type = 'WITHDRAW'
       AND NEW.Amount > CurrentBalance THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Withdrawal failed: Insufficient balance';
    END IF;
END //
DELIMITER ;
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'WITHDRAW', 1000000);
```

Output:



Procedure – Account Transfer

```
DELIMITER //
CREATE PROCEDURE TransferMoney(
    IN SenderAccount INT,
    IN ReceiverAccount INT,
    IN TransferAmount DECIMAL(12,2)
)
BEGIN
    DECLARE SenderBalance DECIMAL(12,2);
    SELECT Balance
    INTO SenderBalance
    FROM Account
    WHERE Account_No = SenderAccount;
    IF SenderBalance < TransferAmount THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transfer failed: Insufficient balance';
    ELSE
        UPDATE Account
        SET Balance = Balance - TransferAmount
        WHERE Account_No = SenderAccount;
        UPDATE Account
        SET Balance = Balance + TransferAmount
        WHERE Account_No = ReceiverAccount;

    END IF;
END //
DELIMITER ;
CALL TransferMoney(10001, 10002, 5000);
SELECT *
FROM Account
WHERE Account_No IN (10001,10002);
```

Output:

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	38000.00	Hyderabad
	10002	102	Savings	80000.00	Vijayawada
*	NULL	NULL	NULL	NULL	NULL